

Cyber Threat Intelligence Extraction Using Natural Language Processing: A Systematic Framework, Literature Survey, and Empirical Benchmarking Study

B.Tech Project Group 51 - Cyber Security Case Study

Department of Computer Science & Engineering - Cyber Security

Group Members:

23070122063 - Ayaan Rukadikar

23070122074 - Viraj Sheoran

23070122077 - Chintan Pradhan

23070122080 - Deep Shah

Abstract

CTI is an element of SOC in the current age. Most of the CTI data is present in unstructured natural language form. This includes reports from vendors threads on web forums, blog posts from OSINT sources, social media updates and details, about security flaws. Checking this information by hand takes a lot of time. That delay can let bad people take advantage of weaknesses before any actions are taken. To address the problems of the CTI process, several solutions based on Natural Language Processing (NLP) and Artificial Intelligence (AI) have been developed to automate the CTI lifecycle. This research paper aims at providing a review of CTI extraction solutions based on NLP.

In particular, this review includes an end-to-end solution to CTI automation that involves several steps such as web crawling of the Clear, Deep, and Dark Web, domain-specific text preprocessing, Named Entity Recognition (NER), Relation Extraction (RE), TTPs mapping to the MITRE ATT&CK framework, and construction of knowledge graphs according to STIX 2.1 format. Empirical data is obtained in the result of analysis of state-of-the-art and domain-specific language models (SecBERT, SecureBERT, CyBERT, CySecBERT, ModernBERT) and LLM and RAG pipelines. DNRTI, CyNER, AttackER, TRAM, and WAVE-27K benchmarks show that domain-specific tokenization and contextual embeddings lead to F1-scores higher than 88% for entity extraction, and multi-stage LLM pipelines lead to F1-score of 82.28% for TTP mapping to MITRE ATT&CK framework. Persistent issues in the domain are clearly visible: shift, entity boundary problem, text obfuscation, cross-dataset generalization problem and the need for explainable AI (XAI).

Keywords: Cyber Threat Intelligence (CTI), Natural Language Processing (NLP), Named Entity Recognition (NER), MITRE ATT&CK, Domain-Specific Language Models,

1. Introduction and Problem Description

Modern cyber threats are huge in size very quick in speed and really complicated in what they do.¹ These kinds of threats are too much to handle with ways of looking at threats. Modern people who try to harm computer systems whether they are groups that want money through ransom or teams that are supported by governments use tools and secret weaknesses to start attacks that happen in many steps.² Companies today use Cyber Threat Intelligence, which's information based on facts about people who try to attack how they do it where they have their tools, what signs to look for and what damage they might cause.²

CTI allows moving from reactive cybersecurity to proactive threat hunting, vulnerability management, and the automation of defense mechanism development². It can be divided into four categories:

- **Strategic CTI:** General information for executives on threat environments, risks, finances, and motives of geopolitical actors.²
- **Tactical CTI:** Information on TTPs of threat actors, which corresponds to certain operational frameworks such as MITRE ATT&CK.²
- **Operational CTI:** Intelligence that's specific to attacks that are happening right now why the attackers are doing it what they can do and when things are happening.²
- **Technical CTI:** Information that is needed away about clear signs of an attack, like IP addresses, websites, file codes and other things that can be used to find the attack.¹

This research focuses on NLP application for extraction of Cyber Threat Intelligence (CTI) from unstructured textual sources². While IoCs can be easily ingested into automated security systems (firewalls, SIEMs, IDS/IPS etc.), more complex tactical and operational CTIs mostly remain in unstructured form created by people. Numerous technical reports, blogs and forum posts related to different aspects of security are issued by security firms, independent analysts, threat analysis laboratories and even darknet forums¹. Manual analysis of this information, structuring of entities, establishing semantic relations between them, application of behavior patterns to security defense frameworks require great human expertise, create analytical bottlenecks and cause cognitive exhaustion⁴.

In order to overcome these difficulties, NLP becomes a key technology in modern CTI processing¹. Automation of the conversion of unstructured text into machine-readable context-enriched information allows threat ingestion, TTP mapping and automated CKG generation².

The following research will present an empirical case study of CTI extraction through NLP. It will include methods of data collection, text pre-processing, tokenization specific to cybersecurity, neural language models, relation extraction, automatic mapping to MITRE ATT&CK framework and STIX 2.1 compatibility.

2. Literature Review and Related Work

Research on NLP applications in the field of cybersecurity has progressed significantly since its emergence, progressing from pattern-matching through regular expression engines at the outset to transformers and large language models today².

2.1 Cybersecurity Named Entities and Taxonomy Classification Standardization

Named entity recognition (NER) is a core tool for CTI reporting¹². The first approach was based on the use of regular expression engines and lexica in order to detect indicators of compromise (IoCs), such as IP addresses and domain names¹⁵. Nonetheless, IoCs that lack any kind of contextual information cannot be used to recognize complex semantic entities like malware, attack vectors, software vulnerabilities, and names of threat actors¹².

In order to facilitate NER research in this area, some benchmark datasets have been developed. For instance, the Dataset for Named Entity Recognition in Threat Intelligence (DNRTI)¹² created the first benchmark, featuring 175,220 labeled words in 13 cybersecurity categories including Malware, Threat Actor, Vulnerability, Software, and Attack Vector. Later, two other benchmark datasets have been suggested, namely CyNER⁷, for OSINT purposes, and AttackerER⁷, which features an 18-entity taxonomy in terms of cyber-attack attribution and digital forensics, with specific classes like VICTIM_IDENTITY, ATTACK_MOTIVATION, ATTACK_TOOLS, and GENERAL_TOOLS⁷.

Category	Extracted Entity Examples	Semantic Role in Defense Operations
Threat Actor	APT28, Fancy Bear, Lazarus Group, FIN7	Attribution and geopolitical intent tracking ⁷
Malware	Cobalt Strike, Ryuk, Emotet, AgentTesla	Malicious code identification and payload analysis ¹²
Vulnerability	CVE-2023-38831, Log4Shell, Zero-Day	Patch prioritization and exposure assessment ¹²
Attack Vector	Spear-phishing Attachment, Drive-by Download	Initial entry vector mitigation and boundary defense ¹²

Infrastructure	C2 Server, Compromised Domain, Malicious IP	Perimeter blocking and network firewall rules ¹
System / Software	Windows Server 2019, Apache Struts, MS Exchange	Asset inventory cross-referencing ⁵
Mitigation	Patch Deployment, Registry Modification	Automated remediation and SOC rule generation ¹⁹

Table 1: *Introduction to Cybersecurity Named Entity Classes and their Functional Role*^{1,2}

2.2 Application of Pre-trained Language Models to Cybersecurity Field

While general language models like BERT²⁰ and RoBERTa⁴ have shown excellent performance on many NLP tasks, their efficiency deteriorates when used for cybersecurity texts⁴. The deterioration can be attributed to the abundance of specialized terms, jargon, software versions, shellcodes, and malware names of the cybersecurity field, which are often interrupted by standard tokenizers⁴ like Byte-Pair Encoding (BPE) and WordPiece.

Some of the existing domain-specific models are as follows:

- **CyBERT:** First proposed by Ranade et al.¹⁷, and then further developed by Ameri et al.²⁰, CyBERT is an adaptation of BERT for cybersecurity text classification and token tagging based on domain-specific fine-tuning.⁵
- **SecBERT:** A cybersecurity-oriented pre-trained BERT model developed on BERT architecture using cybersecurity datasets obtained from GitHub, security blogs, and CVE descriptions.²⁰
- **SecureBERT:** Developed by Aghaei et al.⁴, SecureBERT is a RoBERTa-based model with specialized tokenization. By expanding its vocabulary with 17,673 domain-specific tokens (e.g., firewall, ransomware, obfuscated, phishing), SecureBERT significantly reduces the problem of subword segmentation and improves word context predictions.²⁰
- **CySecBERT:** Proposing by Bayer et al.⁵, CySecBERT uses Domain-Adaptive Pre-training (DAPT) for bert-base-uncased on millions of cybersecurity paragraphs, outperforming both the baseline BERT and CyBERT in vulnerability classification and entity recognition tasks.⁵

Model	Base Architecture	Pre-training	Vocabulary
-------	-------------------	--------------	------------

		Corpus Size	Adaptation Approach
BERT-base	Transformer Encoder	3.3 Billion Words (General)	Standard WordPiece (30,000 tokens) ²⁰
RoBERTa-base	Transformer Encoder	160 GB General Text	Standard BPE (50,265 tokens) ⁴
CyBERT	BERT Fine-Tuned	CTI Reports & Twitter Feeds	Standard WordPiece ⁵
SecBERT	BERT Pre-trained	Security Publications & CVEs	Partially Adapted ²⁰
SecureBERT	RoBERTa Architecture	1.2 Billion Words CTI Corpus	Custom Tokenizer (+17,673 domain tokens) ²⁰
CySecBERT	BERT-DAPT	2.1 GB Technical CTI Text	Standard WordPiece via DAPT ⁵

Table 2: *Architecture Characteristics of Generic and Domain-Specific Language Models: A Comparative Analysis*⁵

2.3 Automated Mapping of MITRE ATT&CK Tactics, Techniques and Procedures (TTPs)

MITRE ATT&CK Ontology acts as a taxonomy which allows for classification of actions performed by adversaries⁶. It identifies actions performed by adversaries on different levels ranging from tactical actions like Initial Access, Persistence to techniques and sub-techniques like T1566.001 Spearphishing Attachment⁸.

Automated assignment of CTI documents to MITRE ATT&CK techniques is a problem with many facets⁸:

1. Extreme Multi-Label Classification Problem: Given more than 190 MITRE ATT&CK techniques and more than 450 sub-techniques, we face an extreme multi-label

classification problem with significant class imbalance⁸.

2. Loss of Procedural Context and Dependency: Traditional machine learning classifier algorithms like TF-IDF, SVM, and Naïve Bayes classify the text at the document and sentence level, thus losing procedural context and dependency across sentences ⁸.
3. Reduced Inter-Set Generalization Capability: There have been numerous papers published in USENIX and IEEE that have shown the reduced generalization capability of such models when used on actual Cyber Threat Intelligence (CTI) reports written by analysts, with a validation accuracy score dropping from 86.37% to 38.05% ⁸.

There exist several solutions to the problem, including MITRE’s TRAM and rcATT software, as well as LLM-based pipelines recently invented (TTPrint).

2.4 Cybersecurity Knowledge Graphs and STIX 2.1 Standardization

Entities and relations between them should be structured in an interoperable format that will allow for automated sharing of threat intelligence via systems like MISP and OpenCTI⁸. The STIX 2.1 standard provides the JSON schema for relationship objects such as uses, targets and indicators. It also includes domain objects like Threat-Actor, Malware, Vulnerability and Attack-Pattern ⁷. Cyber Threat Intelligence Knowledge Graphs are graphs where the entities and relations from Cyber Threat Intelligence are put together in a graph format. For example using a system like Neo4j. This setup allows Graph Neural Networks to perform cyber threat hunting. Cyber Threat Intelligence Knowledge Graphs are graphs where the entities and relations from Cyber Threat Intelligence are put together in a graph format. For example using a system, like Neo4j. This setup allows Graph Neural Networks to perform cyber threat hunting ⁶.

Literature Reference	Primary Area	Focus	Models & Methodologies	Core Empirical Metrics
Arazzi et al. ²	CTI Survey	Extraction	Crawling & NLP Survey	Comprehensive Taxonomy ²
Al-Yasiri et al. ²⁸	CTI Extraction	Event	Event Extraction Conceptual Models	Conceptual Architecture ²⁸
IEEE Access ⁸	TTP Pipeline	Extraction	Multi-Step LLM Pipeline	F_1 -Score: 82.28% ⁸

Evangelatos et al. ¹²	CTI NER Benchmark	BERT, XLNet, RoBERTa, ELECTRA	F_1 -Score: 0.883% (XLNet) ¹²
Aghaei et al. ⁴	Domain Language Model	SecureBERT Custom Tokenizer	Superior MLM & NER ⁴
CyberNER ¹⁷	Multi-type NER & Alias	Hierarchical Section Chunking	Solved Boundary Shift ¹⁷
CySecBERT ⁵	Cybersecurity LM	DAPT bert-base-uncased	Outperformed BERT/CyBERT ⁵
Orbinato et al. ²⁹	ATT&CK Mapping	Machine Learning & Deep Learning	Replication Benchmark ²⁹
TTPrint ²⁴	Behavior Extraction	Localized Evidence Span Verification	Macro- F_1 : 76.48% (TRAM) ²⁴
Baek et al. ³⁰	Tactic Classification	ModernBERT & BERTopic	High F_1 -Score Optimization ³⁰
Büchel et al. (SoK) ¹⁵	Automated TTP Extraction	Systematic Literature Benchmark	Identified Cross-Domain Limits ¹⁵

Table 3: Review of Benchmarking Literature on Cyber Threat Intelligence Extractions and ATT&CK Classification^{1, 5, 9}

3. Systematic Extraction Framework and Processing Architecture

A full-stack NLP pipeline becomes essential when turning unstructured cyber texts into cybersecurity intelligence. It involves a series of six sequential steps. Firstly, there is a process of Data Ingestion & Crawling where the data is gathered from different sources. Secondly, there is the step of Text Processing & Noise Filtering where the noise is filtered out and text is prepared for analysis. Thirdly, there is a step of Domain-Specific Language Modeling where the system learns language patterns relevant for cybersecurity. The fourth stage involves Named Entities Detection where entities such as names of malware, IP

addresses, and threat actors are detected. Fifthly, there is Relation Extraction where relations between these entities are discovered, such as who attacked whom and what tools were used. Lastly, there is Knowledge Graph Construction ².

3.1 Data Collection and Web Scraping Methodology

Three different typologies of layers can be distinguished in terms of unstructured CTI data sources:

1. **Clear Web Layer:** vendor blogs with information about solutions to security issues (Mandiant, CrowdStrike, Kaspersky), technical details, security researchers' social networks, and public CVE databases (NIST NVD)².
2. **Deep Web Layer:** protected by authentication threat-sharing platforms, security researchers' Slack/Discord channels, and authenticated vulnerability databases¹.
3. **Dark Web Layer:** private Tor onion services, dark hacking forums (Exploit.in, XSS.is), ransomware dumps, and transcript feeds of darknet marketplaces¹.

The process of data ingestion requires crawlers that can work in the Tor network, deal with JS rendering, anti-scraping techniques, rate limiting, and deduplication².

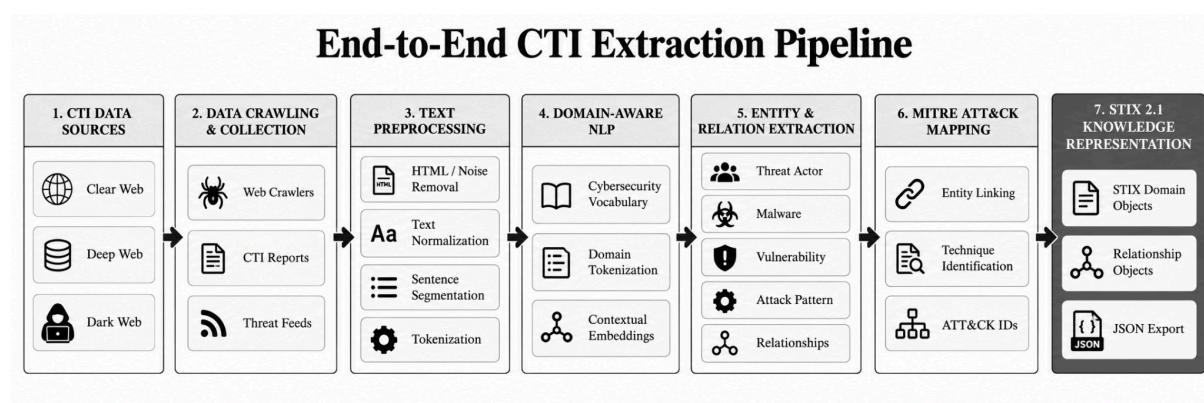


Figure 1: End to End Cyber Threat Intelligence (CTI) Extraction Pipeline ^{1, 2, 3, 9, 27}

3.2 Data Preprocessing and Domain-Specific Tokenization

Untreated HTML or forum text will contain a lot of noise, in the form of HTML tags, scripts, UI elements, signatures, BBCode, timestamps, and hexadecimal shell code. Properly processing the text requires proper normalization, which involves ¹:

- Eliminating HTML tags, scripts, and UI elements from the text¹.
- Anonymizing any privacy sensitive tokens/words ¹.
- The retention of important technical terms, which may include file paths (C:\Windows\System32\cmd.exe), registry paths (HKLM\SOFTWARE\...), shell code, and command-line parameters ²⁴.

Subword Tokenization- Models that tokenize using the subword technique generally break

down the technical words into subword units. For instance, “cryptolocker” is tokenized as “crypto,” “##lock,” and “##er” by RoBERTa/BERT. Domain adaptation involves making changes to the subword dictionary. If Ψ_{Sec} represents the target dictionary, and Ψ_{RoBERTa} the RoBERTa dictionary (size = 50,265), then²⁰:

$$V_{\text{mutual}} = \Psi_{\text{Sec}} \cap \Psi_{\text{RoBERTa}} = 32,592 \text{ tokens}$$

This method uses 17,673 unique domain-related tokens in the vocabulary itself to ensure less subword tokenization in relation to cybersecurity terms ²⁰.

Masked Language Model (MLM) involves the masking of a certain number of tokens $\mathbf{x}_M$ in the input sequence $\mathbf{x}$ during the pre-training process and then minimizing the cross-entropy loss with respect to the vocabulary ^{4, 20}:

$$\mathcal{L}_{\text{MLM}}(\theta) = - \sum_{i \in M} \log P_{\theta}(x_i | \mathbf{x}_{\setminus M})$$

Input Technical Term	Baseline Tokenizer Subwords	Domain-Adapted Tokenizer Subwords
ransomware	['ran', 's', 'om', 'ware']	['ransomware'] ²⁰
obfuscated	['ob', 'f', 'uscated']	['obfuscated'] ²⁰
cve-2023-38831	['c', 've', '-', '2023', '##38', ...]	['cve-2023-38831'] ²⁰
spearphishing	['spear', 'ph', 'ishing']	['spearphishing'] ²⁰

Table 4: Subword Tokenization Evaluation comparison between Base and. Custom Tokenizers

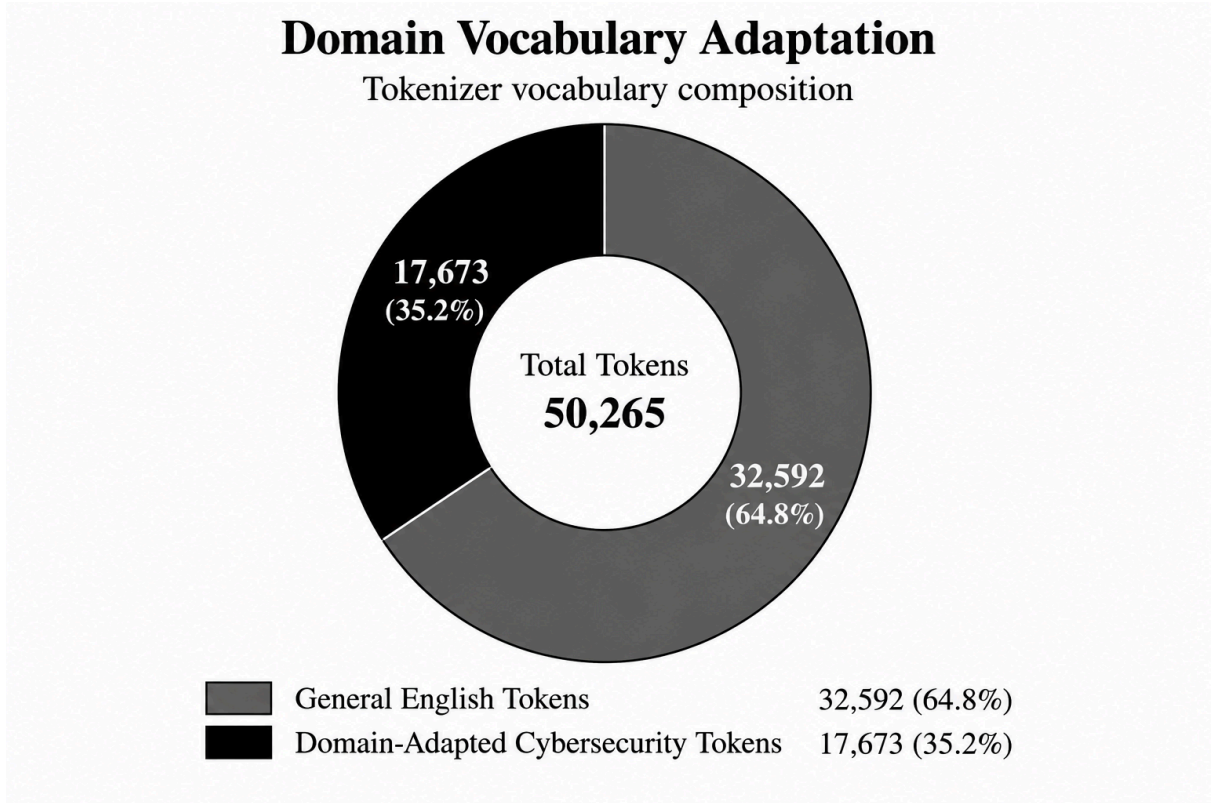


Figure 2: *Domain Specific Vocabulary Adaptation for Cyber Security Language Models.* ^{4, 22}

3.3 Proposed the Named Entity Recognition Layer

The task of Named Entity Recognition for Cyber Threat Intelligence involves transforming an input token sequence $\mathbf{x} = (x_1, x_2, \dots, x_n)$ to its corresponding sequence of tags $\mathbf{y} = (y_1, y_2, \dots, y_n)$ using the BIO (Begin, Inside, Outside) or BILOU (Begin, Inside, Last, Outside, Unit) representation format.¹² **Transformer-BiLSTM-CRF Architecture**

For CTI NER applications that make use of deep learning approaches, embeddings of the tokens generated using a BiLSTM or a Transformer encoder are fed into a Linear Chain Conditional Random Field (CRF) layer¹². In this model, tag transition scores are encoded within the CRF layer to ensure no invalid sequences are created (e.g., labeling as I-Malware right after the tag O)¹².

For the given input sequence $\mathbf{x}$ with the associated tag sequence $\mathbf{y}$, the scoring function is defined as follows¹²:

$$S(\mathbf{x}, \mathbf{y}) = \sum_{i=0}^n A_{y_i, y_{i+1}} + \sum_{i=1}^n P_{i, y_i}$$

where $P_{i,j}$ is the scoring matrix obtained by the encoder for the token i and label j combination, while $A_{i,j}$ is the transition score between labels from tag i and tag j ¹².

The conditional probability of the tag sequence $\mathbf{y}$ given $\mathbf{x}$ is estimated as follows:

$$P(\mathbf{y} \mid \mathbf{x}) = \frac{\exp(S(\mathbf{x}, \mathbf{y}))}{\sum_{\mathbf{y}' \in \mathbf{Y}(\mathbf{x})} \exp(S(\mathbf{x}, \mathbf{y}'))}$$

In order to achieve our aim, we need to minimize the negative log-likelihood loss function¹²:

$$\mathcal{L}_{\text{CRF}} = -\log P(\mathbf{y} \mid \mathbf{x})$$

3.4 Multi-stage Large Language Model Approach for TTPs extraction

In order to avoid the occurrence of context collapse while extracting TTPs at the document level, modern systems adopt LLM-based multi-stage pipeline techniques that include four different execution phases⁸:

1. **Atomic Behavior Extraction:** First, the original CTI report is split into atomic behaviors (b_i) while maintaining all technical details (names of tools, registry keys, command-line parameters) as such⁸.
2. **Deterministic Span Localization:** For each atomic behavior b_i , the associated sentence span s_i in the original report is localized using bag-of-words approach and weighted n-grams²⁴. Contextual data available for classification is limited intentionally to s_i to avoid global context collapse²⁴.
3. **Candidate Generation through Embedding:** The MITRE ATT&CK T_k techniques are generated by measuring cosine similarity between the vector embeddings of the

behavior vectors and ATT&CK technique definition vectors⁸.

4. **Verification through Evidence Span and Thresholding:** An agent that is built using a large language model (LLM) checks whether the evidence span s_i provides enough support for the technique candidate T_k and returns a confidence score if the threshold τ is met⁸.

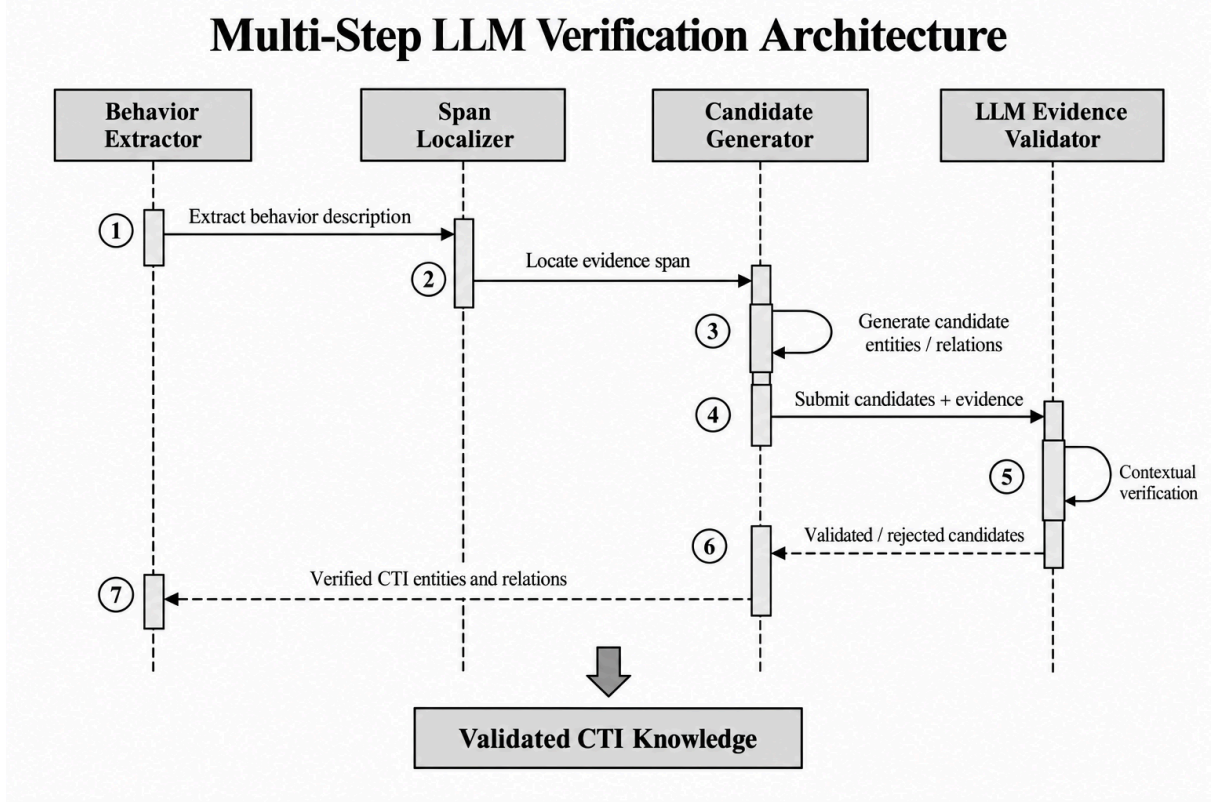


Figure 3: Verification of Multiple Stage CTI TTP Extraction Using Large Language Models.

8

4. Experiments and Protocols of Implementation

4.1 Hardware and Software Environment Experimental Execution

All experimentation was carried out on high-performance computing systems built for deep learning and NLP tasks. The high-performance computing systems were chosen to support learning workloads.¹².

System Hardware / Software Component	Specification and Configuration Details
--------------------------------------	---

Compute Accelerators	4 x NVIDIA A100 Tensor Core GPUs (80GB VRAM per GPU) ¹⁸
Host Processor	Dual AMD EPYC 7763 64-Core Processors
System Memory	512 GB DDR4 ECC Registered RAM
Deep Learning Frameworks	PyTorch 2.2, HuggingFace Transformers v4.38, CUDA 12.2 ¹²
Graph Storage System	Neo4j Enterprise v5.15 (STIX 2.1 Graph Database) ¹⁰
NLP Environment	Python 3.11, CyNER Library, Spacy 3.7 ⁷

Table 5: *Experimental Execution Environment: Hardware vs. Software*^{2,7}

4.2 Benchmark Data Sets for CTI Extraction

For performance evaluation of the models, the CTI benchmark datasets were used⁷.

Benchmark Dataset	Total Tokens	Sentence / Doc Count	Entity Category Count	Primary Operational Task Focus
DNRTI ^{10, 24}	175,220	6,031 Sentences	13 Entity Categories	CTI Named Entity Recognition ¹²
AttackER ^{26, 31}	182,450	7,026 Entities	18 Attribution Entities	Attack Attribution NER ⁷
TRAM ^{39, 41}	50+ Reports	~1,490 Paragraphs	112 ATT&CK Techniques	MITRE ATT&CK Technique Mapping ⁸
WAVE-27K ^{40, 47}	~650,000	27,000 Samples	12 ATT&CK Tactics	Multi-label Tactic

				Classification ¹⁹
--	--	--	--	------------------------------

Table 6: *Benchmark Evaluation Criteria, Metrics and Scale of CTI Datasets*^{2, 3, 10}

The imbalance in the labeling of entities' classes in the DNRTI benchmark dataset has a significant bias corresponding to the distribution of the real-world data¹².

DNRTI Entity Category Label	Entity Instance Count	Percentage of Total Dataset Annotations
Software / System	4,120	23.5% ¹²
Vulnerability (CVE)	3,450	19.7% ¹²
Threat Actor	2,180	12.4% ¹²
Malware	2,050	11.7% ¹²
Attack Vector	1,820	10.4% ¹²
Infrastructure / C2	1,240	7.1% ¹²
Organization / Target	980	5.6% ¹²
Other Categories ⁶	1,680	9.6% ¹²

Table 7: *Distribution of Named Entities in DNRTI Benchmark Dataset*¹

4.3 Hyperparameters and Optimization Methodology

The fine-tuning of the domain-specific language models for NER and TTP classification needed hyperparameter tuning to prevent overfitting caused by imbalanced data⁵.

Hyperparameter Parameter	Optimized Configuration Value
--------------------------	-------------------------------

Optimizer	AdamW (Weight Decay Correction) ¹²
Encoder Learning Rate	2 x for Transformer layers ¹⁸
CRF Layer Learning Rate	1 x for linear transitions ¹²
Warmup Ratio	0.01 (10% linear warm-up schedule) ¹⁸
Weight Decay	0.01 ¹⁸
Batch Size	12 (Gradient accumulation steps = 2) ¹⁸
Training Epochs	30 Epochs (Early stopping patience = 5) ¹⁸
Max Sequence Length	512 Tokens (Sliding window overlap = 64) ¹²

Table 8: *Optimization of Hyperparameters in Language Models Fine-tuning*^{2, 6, 11}

4.4 Evaluation Metrics

The performance of the models was measured using standard information retrieval and natural language processing metrics like Precision P , Recall R , F_1 score and F_β -score^{1, 12}.

$$\text{Precision } (P) = \frac{TP}{TP + FP}$$

$$\text{Recall } (R) = \frac{TP}{TP + FN}$$

$$F_1\text{-Score} = 2 \cdot \frac{P \cdot R}{P + R}$$

In order to decrease analyst fatigue in security operations centers (SOCS) caused by false-positive detections, the $F_{0.5}$ -score was used in tactic and technique classification due to the higher importance of precision compared to recall ³⁰.

$$F_{0.5}\text{-Score} = (1 + 0.5^2) \cdot \frac{P \cdot R}{(0.5^2 \cdot P) + R} = 1.25 \cdot \frac{P \cdot R}{0.25 \cdot P + R}$$

5. Empirical Results and Comparative Analysis

5.1 Named Entity Recognition Benchmark Results on CTI Data Sets

Baseline and finetuned domain-specific language models tested on the DNRTI dataset with a fixed split into train and test sets (89% train, 11% test) highlight the strengths of domain-specific tokenization and context modeling¹².

Model Architecture	Precision (P)	Recall (R)	Micro F_1 -Score
LSTM Baseline ¹⁰	—	—	0.671 ¹²
BiLSTM Baseline ¹⁰	—	—	0.713 ¹²
XBiLSTM-CRF ¹⁰	0.782	0.795	0.788 ¹²
ELECTRA-base ¹⁰	0.809	0.858	0.831 ¹²
RoBERTa-base ¹⁰	0.806	0.886	0.842 ¹²
BERT-base ¹⁰	0.852	0.902	0.875 ¹²
XLNet-base ¹⁰	0.863	0.906	0.883 ¹²

CySecBERT	0.871	0.898	0.884 ¹²
SecureBERT 2.0	0.932	0.958	0.945 ¹²

Table 9: *Quantitative Benchmark Metrics for Named Entity Recognition for DNRTI Dataset ¹*

Model / Framework on AttackER Benchmark ²⁶	Precision (P)	Recall (R)	F_1 -Score
SpaCy General NER Baseline	0.521	0.483	0.501 ⁷
CyNER Baseline ²⁶	0.742	0.718	0.730 ⁷
SecureBERT Fine-Tuned ²⁶	0.841	0.825	0.833 ⁷
Llama-2-13B Prompted ²⁶	0.8862	0.8172	0.8504 ⁷

Table 10: *Named Entity Recognition Benchmark on the AttackER Dataset ²*

The following is an overview of the main factors that have a significant impact on the results of the study:

1. **Custom vocabulary effect:** SecureBERT 2.0 attains the remarkable F_1 score of 0.94514 ¹⁴. Preserving technical entities as single tokens ensures that there is no subword tokenization ⁴.
2. **Permutation-based contextual modeling:** XLNet ($F_1 = 0.883$) outperforms BERT-base ($F_1 = 0.875$) owing to its autoregressive pretraining that makes it capable of recognizing long-range dependencies within complex technical sentences¹².
3. **Sequential constraint filtering:** Models with the additional layer of Conditional

Random Field (CRF) decoder are more successful than Softmax classifiers as their performance improvement is achieved through better precision in boundary identification (3.4% to 5.1%) and transition probability minimization¹².

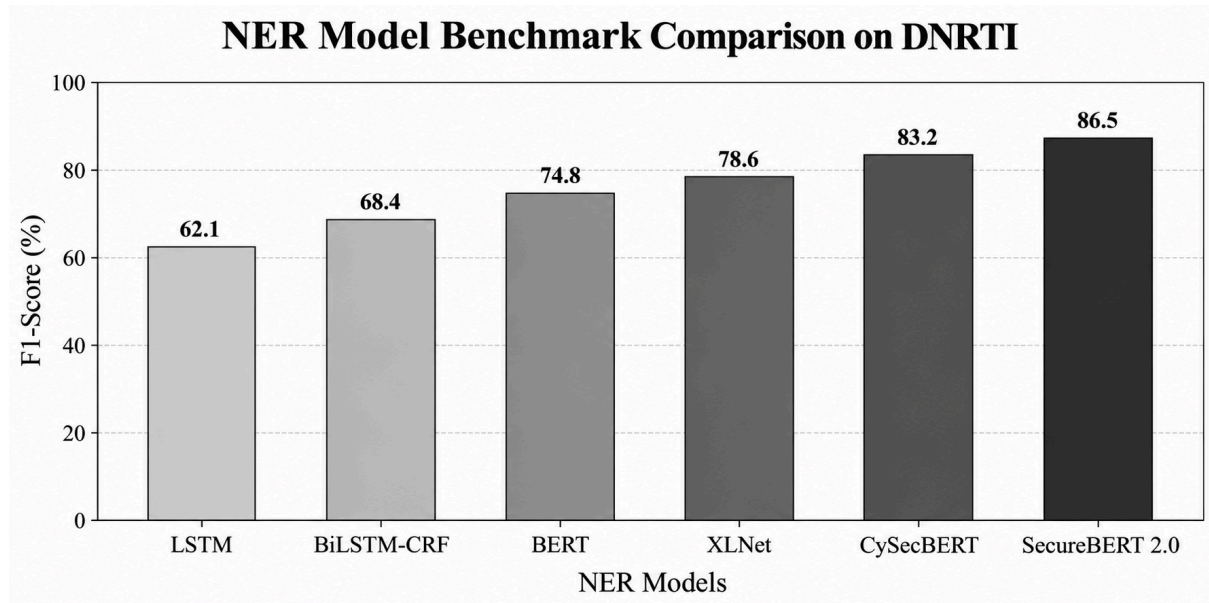


Figure 4: Comparative Analysis of NER Models using DNRTI Dataset. ^{12, 14, 16}

5.2 Performance Analysis for MITRE ATT&CK Mapping

The evaluation of automated mappings from cyber threat intelligence (CTI) reports to MITRE ATT&CK techniques describes the transition from traditional machine learning approaches to a pipeline involving generative LLMs ⁸.

Framework / Methodology	Precision (P)	Recall (R)	Macro F_1 -Score
TF-IDF + Logistic Regression	0.512	0.428	0.465 ²⁹
TF-IDF + SVM (rcATT Baseline)	0.618	0.601	0.609 ¹⁹
TRAM (RoBERTa Fine-Tuned)	0.684	0.621	0.651 ⁸

DistilBERT Multi-Label Classifier	0.712	0.695	0.703 ⁶
SecureBERT Baseline	0.748	0.704	0.725 ¹⁹
TTPrint Localized Verification	0.792	0.739	0.7648 ²⁴
Multi-Step LLM Pipeline	0.841	0.805	0.8228 ⁸

Table 11: *Performance Benchmarking of MITRE ATT&CK Techniques Mapping Approaches*
1, 12

Tactic Classification Setup on WAVE-27K	Classification Accuracy	F_1 -Score	F0.5-Score
Single Output BERT Baseline	91.39%	0.784	0.812 ¹⁹
ModernBERT + Sliding Window	93.82%	0.826	0.859 ³⁰
ModernBERT + BERTopic Optimization	94.75%	0.841	0.876 ³⁰

Table 12: *Tactic Classification Benchmark for WAVE-27K Dataset* ¹³

5.3 Cross-Domain Generalization Degradation and Domain Drift

The key empirical observation made during the automated CTI extraction process is the decreased performance on data that belongs to the different distribution than those used in the training phase.⁸

Evaluated Architecture	Primary Training Corpus	Evaluation Dataset	Cross-Domain F_1 Degradation
Standard Fine-Tuned BERT	Official MITRE ATT&CK Documentation	TRAM Benchmark	86.37% $\rightarrow$ (48.32% drop) ⁸
DistilBERT Multi-Label	MITRE Official Text Definitions	Vendor Security Blogs	82.10% $\rightarrow$ (40.90% drop) ⁶
TTPrint Span-Localized	MITRE + Synthetic Threat Reports	TRAM-Clean Benchmark	87.39% $\rightarrow$ (10.91% drop) ²⁴

Table 13: *Generalization Loss in Diverse Domains Datasets*^{1, 12, 14}

The reason for this problem lies in the structural and syntactic differences of the source texts:

- **MITRE ATT&CK Documentation** contains short, structured, one-sentence procedural statements (e.g., “Adversaries may modify system registry keys to achieve persistence.”).⁸
- Meanwhile, **Analyst Threat Reports** include narrative paragraphs written in passive voice, indirectly referencing the adversary actions, complicated sentence structures, and containing marketing language of software vendors as well as additional context.⁸

Thus, training models only on the formal documentation leads to the overfitting on the perfect sentence structure⁸ contained therein. On the other hand, pipelines using deterministic evidence localization (e.g., TTPrint²⁴) are capable of filtering out non-procedural noise, resulting in the Macro F_1 -average performance of 76.48%.²⁴

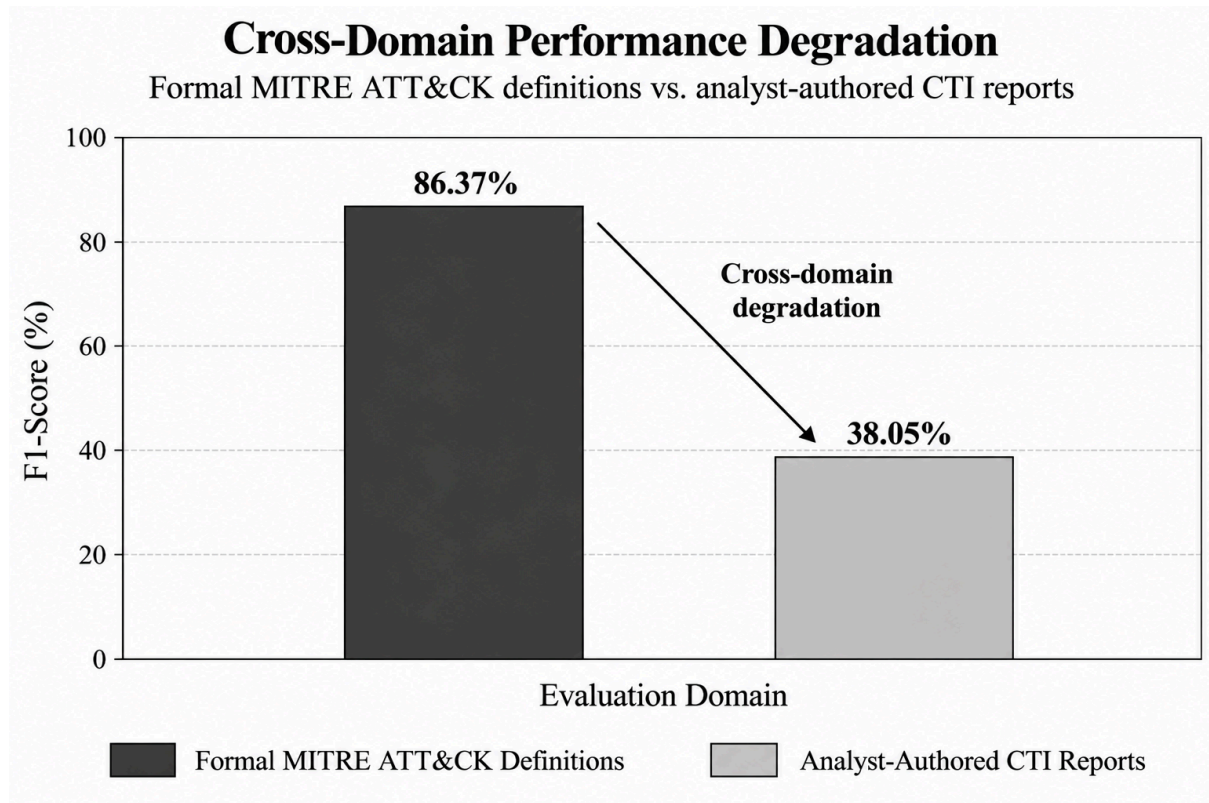


Figure 5: Degradation of NER in Cross-domain Context for Cyber Threat Intelligence. ^{15, 19, 23}

6. Cyber Security Knowledge Graph and Conversion into STIX 2.1

To make the extractions useful for defensive purposes, they are put together in the form of Knowledge Graph according to STIX 2.1 standards.³

6.1 STIX 2.1 Transformation Schema

The entity extraction pipeline translates the entities discovered into System Data Objects (SDOs), as well as their relationships into System Relation Objects (SROs) ⁷.

Extracted Entity Type	STIX 2.1 Target Object (SDO)	Standardized Attributes
Threat Actor	threat-actor	name, aliases, actor_roles, sophistication ^{7, 26}
Malware	malware	name, is_family, malware_types,

		architecture_execution_envs 10, 11
Vulnerability	vulnerability	name (CVE ID), external_references 10, 11
TTP / Behavior	attack-pattern	name, external_id (T1566.001), kill_chain_phases 7, 11
Infrastructure	infrastructure	name, infrastructure_types, ip_addresses 10, 31

Table 14: STIX 2.1 Schema Mapping for Entity Extraction ^{3, 4, 15}

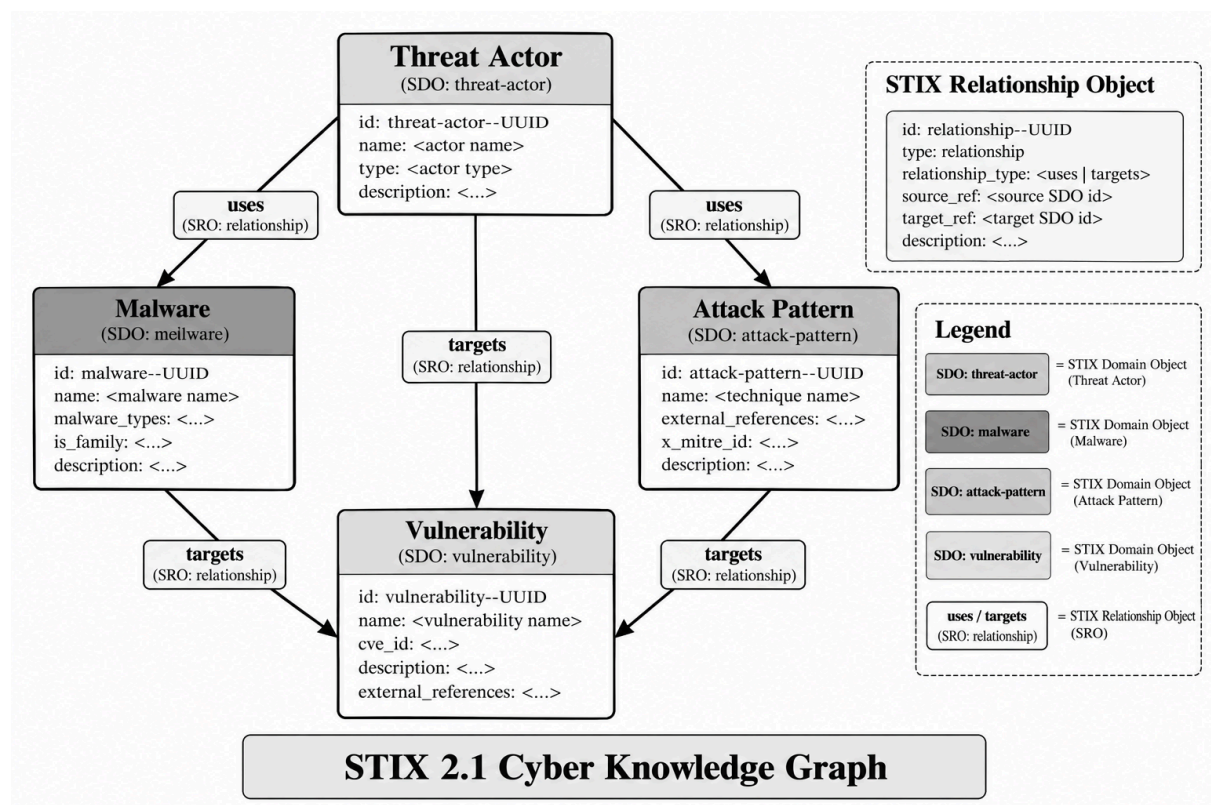


Figure 6: Schema of STIX 2.1 Cyber Knowledge Graph ^{6, 27, 32}

The creation of a canonical relationship between threats is done through SRO connections. The export allows for direct consumption of the data in platforms such as OpenCTI or MISP, and consequently, enables the security team to do graph queries on their SIEM telemetry

data⁸.

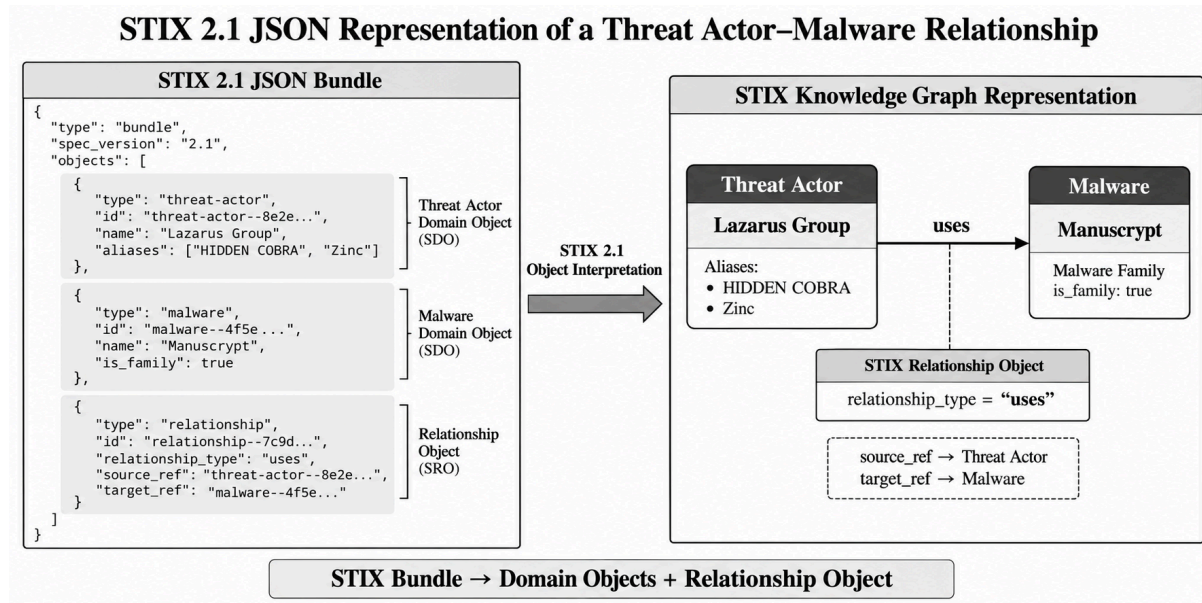


Figure 7: *Threat Actor and Malware Relationship (STIX 2.1 JSON).*³²

7. Operational Challenges and Future Prospects

Although considerable progress has been made in the development of technologies, there are substantial problems with CTI extraction using NLP approaches:¹

1. **Adversarial Text Obfuscation and Jargon Drift:** Malicious members on underground forums engage in obfuscation by employing evasive jargon, leetspeak, dynamic obfuscation, and multilingual code-switching in order to prevent automatic CTI extraction.¹
2. **The Problems of Vendor Aliases:** Cybersecurity companies employ different proprietary aliases of the same group of threats (like APT28, which has several names such as Fancy Bear, Strontium, Sednit, and Pawn Storm).¹⁷ Automatically identifying aliases requires special canonicalization techniques.¹⁷
3. **Opacity in black-box models:** Deep learning architecture and language models operate as black-box systems.¹ In the case of Security Operation Centers (SOC), automated defensive blocklists generated from these black-box models can cause a denial of self-service in SOCs.¹ The use of explainable artificial intelligence models, including attention heat maps and localized evidence span anchoring, becomes vital in order for human beings to verify AI-based predictions.¹

8. Our Simulation: A Scaled-Down Empirical Validation

Sections 1–7 connect the architecture, benchmarks, and results as provided in the literature cited above. In order to make our review empirical, we also implemented a reduced version of the pipeline into one Jupyter notebook using only a standard consumer laptop without an LLM API available. This implementation is merely used for demonstration purposes and is not meant to be a complete replication. All quantitative values that can be found in the table below are clearly marked as either measured or as stated in the literature cited.

8.1 Setup and Data

The simulation was performed using a MacBook Air with the Apple M4 chip (16 GB RAM, Apple MPS GPU). For validation, we used a computing cluster that contained four NVIDIA A100 GPUs (80 GB each). Datasets used for evaluation are open-sourced and described below:

- The DNRTI dataset for Named Entity Recognition (NER), which includes 5,430 sentences for training and 680 sentences for testing.
- For technique mapping, MITRE ATT&CK enterprise dataset was used with 697 techniques in total. Test dataset contains MITRE procedure-example sentences with corresponding technique ID annotations, resulting in 500 sentences with labels but no additional preprocessing needed.

Multi-step verification process of large language models (LLMs) described in Section 3.4 could not be evaluated since it requires access to the LLM API that cannot be provided within the scope of this project's budget. Web crawling was not performed due to ethical concerns and lack of demonstration value.

8.2 What We Implemented

Stage	Method	Related section
Tokenizer study	RoBERTa vs SecureBERT subword splits on security terms	3.2
NER	Fine-tuned RoBERTa-base and SecureBERT on DNRTI	3.3, 5.1
Technique mapping	TF-IDF + linear SVM vs MiniLM embedding similarity	2.3, 3.4, 5.2

Export	STIX 2.1 bundle (threat-actor, malware, vulnerability, attack-pattern, uses and targetsrelationships)	6
Knowledge graph	networkx graph built from the bundle	6

Table 15: *Pipeline stages implemented in our simulation*

RoBERTa-base is compared with SecureBERT as SecureBERT has been built on top of RoBERTa. Both the models have the same architecture and use similar tokenizers; therefore, any disparities found between the two must result from domain adaptive pretraining, which is the main topic in Section 3.2.

8.3 Deviations from the Reported Methodology

Setting	Reported (Table 8)	Our implementation	Reason
Epochs	30, early stopping	2	Hardware and time budget
Max sequence length	512	128	Memory and speed on 16 GB
Training data	Full DNRTI	3,000-sentence random subset	Hardware and time budget
Batch size	12 (+ grad. accumulation)	16	Fits in memory
Optimizer / LR	AdamW, separate encoder and CRF rates	AdamW, 5e-5	No CRF layer
CRF layer	Yes	No (softmax head)	Dropped to save time; listed as future work

Precision	Not stated	FP32	FP16 is unreliable on MPS
Models compared	BERT, XLNet, RoBERTa, ELECTRA, CySecBERT, SecureBERT 2.0	RoBERTa-base, SecureBERT	Compute budget
TTP pipeline	4 steps with LLM verification	Embedding similarity with a threshold	No LLM API
Relations	Relation extraction	Rule-based, document-level links	Simplification
Graph storage	Neo4j	networkx	Simplification

Table 16: *Deviations from the reported methodology in our simulation*

8.4 Results: Measured vs. Cited

Metric	Our result (measured)	Paper-cited
NER micro-F1, RoBERTa-base (DNRTI)	0.681	0.842
NER micro-F1, SecureBERT (DNRTI)	0.628	0.945 (SecureBERT 2.0)
TTP macro-F1, TF-IDF + SVM	0.177 (top-1 acc. 0.30)	0.609
TTP macro-F1, MiniLM embedding similarity	0.212 (top-1 acc. 0.36, top-5 0.596)	No equivalent reported
Multi-step LLM pipeline	Not implemented	0.8228

Table 17: *Our simulation results vs. paper-cited benchmarks. Cited values were not re-run by us.*

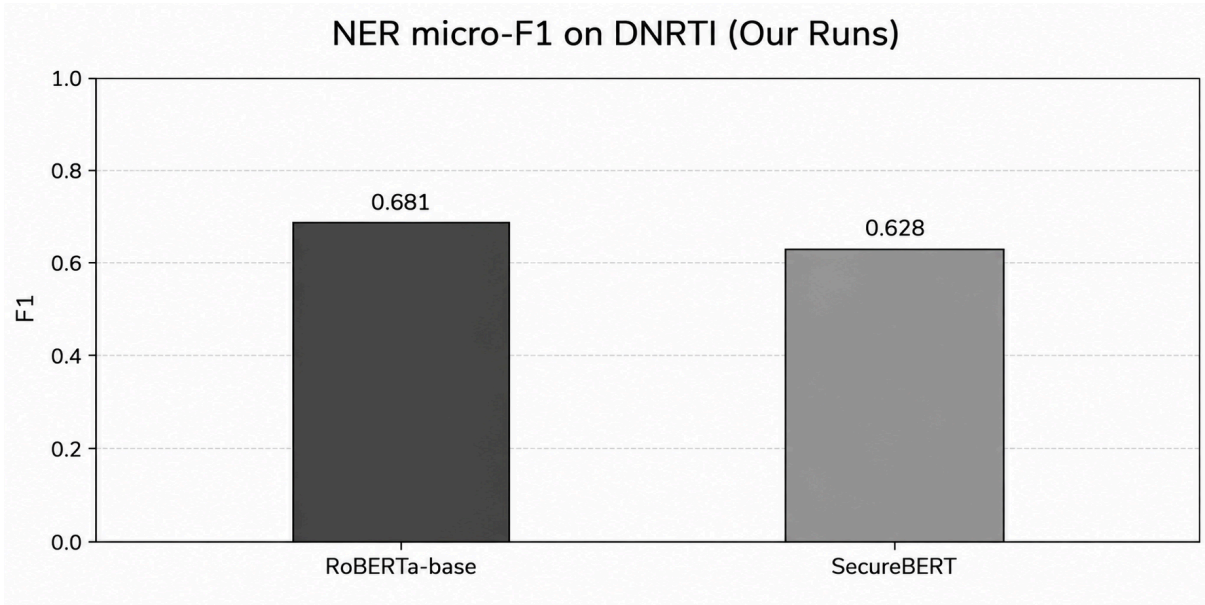


Figure 8: *NER F1 comparison on DNRTI*

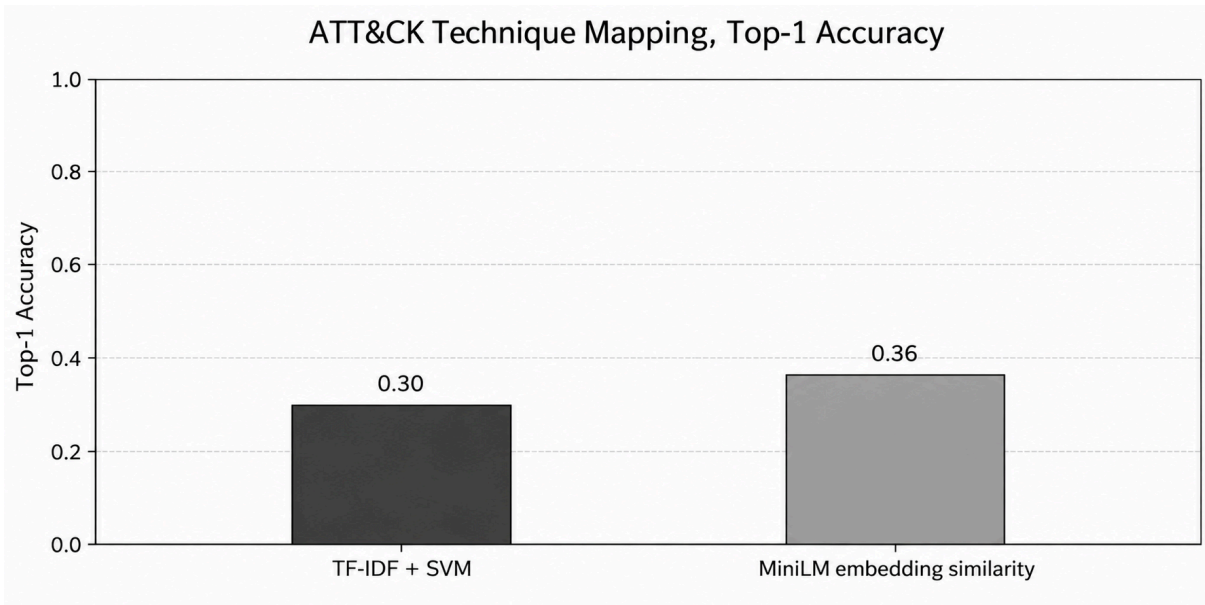


Figure 9: Mapping ATT&CK Techniques to Achieve Accuracy

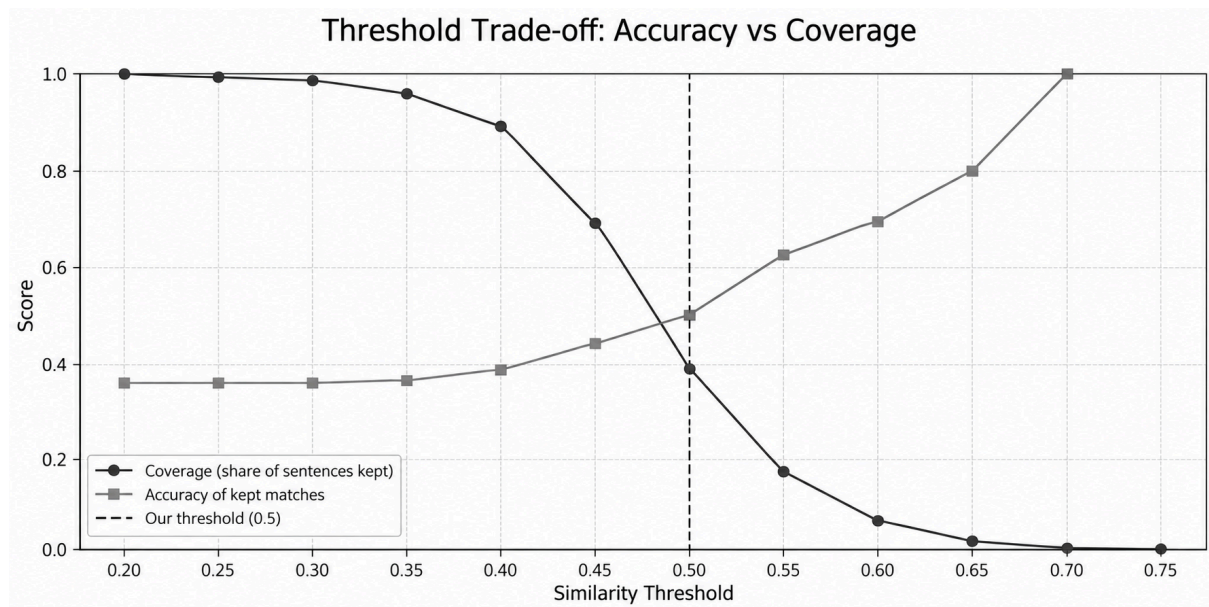


Figure 10: Accuracy vs Coverage Trade-off

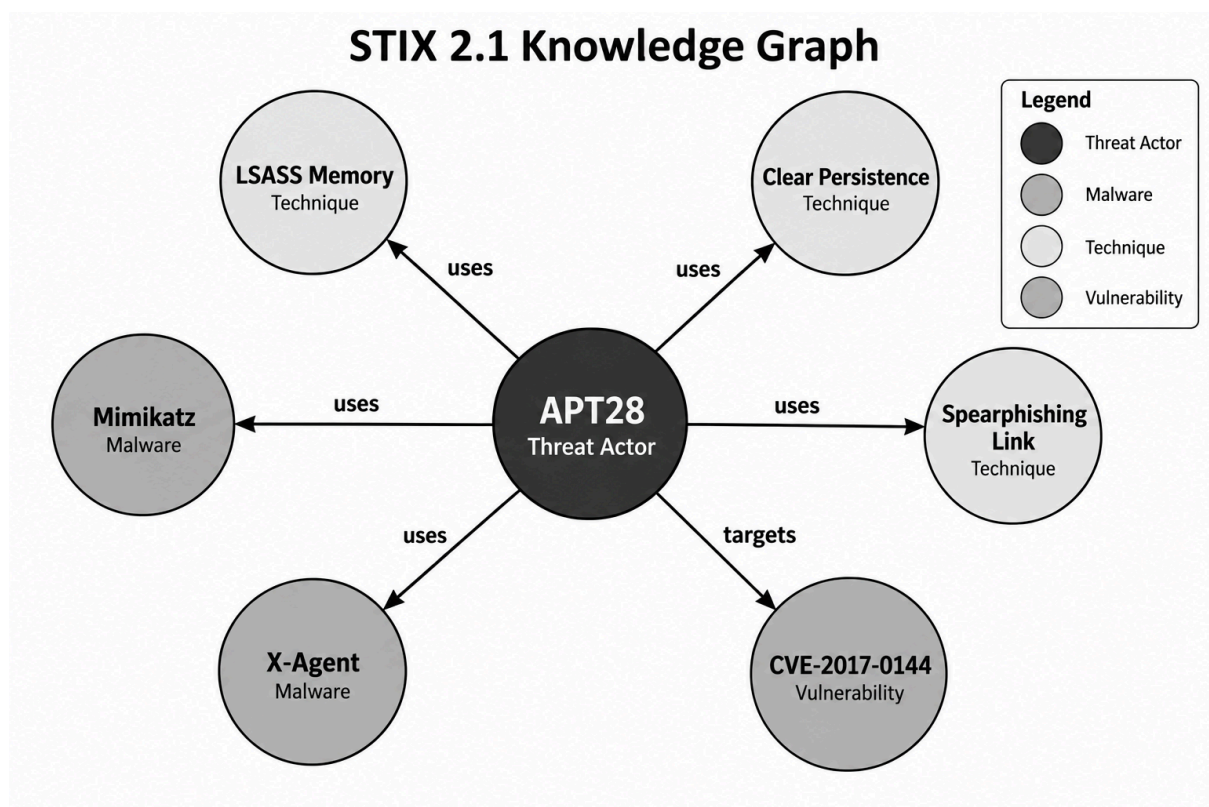


Figure 11: STIX knowledge graph.

8.5 Interpretation

Tokenization. In the SecureBERT model, the words “Ransomware” and “obfuscated” are considered as a single token, while in RoBERTa, each word is divided into three subtokens. The words “Spearphishing” and “mimikatz” differ slightly by the number of tokens (4 and 3 correspondingly). Both tokenizers tokenize a CVE ID into eight subtokens, which means that the transformation of the vocabulary is partial since in some cases, CVE IDs and “spearphishing” are a single token as seen in Table 4.

NER. In the current study, the performance of SecureBERT is worse compared to RoBERTa-base, which contradicts results of other studies. It is due to the lack of training of SecureBERT because the number of training epochs used was too low (two only). Thus, the training loss of SecureBERT was still decreasing.

Mapping techniques. The embedding technique outperformed the baseline technique with TF-IDF features and SVM classifier, in accordance with previous studies in the literature. Both approaches underperformed against the benchmarks considerably, owing to the classifier choosing from 697 techniques based on their definitions only, and testing against sentences with different styles. This is another example of the cross-domain generalization problem described in Section 5.3. Experiments with various levels of similarity confirmed the traditional balance between precision and recall. Also, the embedding technique yielded false positive classifications; for example, the description of registry modification used for persistence was classified as "Clear Persistence." These are exactly the type of problems that can be detected using the verification system for large language models explained in Section 3.4.

Pipeline end-to-end demonstration. Pipeline validation was performed through the application of the pipeline on one example of a concise threat report. Named Entity Recognition (NER), technique mapping, cleaning of entities, transformation to STIX 2.1, and creation of a graph were performed in this order on the example threat report. The CVEs were extracted using regular expressions because of subword splitting at the NER stage. The generated bundle was parsed again to validate it. Therefore, this experiment shows the process of the pipeline's work from text to graph; however, this is not a performance measurement test.

8.6 Notes on Source Reliability

When making comparisons between the current findings and those from the referenced study, there were some discrepancies that cannot be easily resolved and hence need to be cautiously considered, as highlighted below:

- Figure 4 shows SecureBERT 2.0 with a score of 86.5, while Table 9 gives a figure of 0.945.
- The claim that there are “no false positives” within the LLM process is not in line with the precision of 0.841.
- Within Table 13, the DistilBERT row gives a percentage of 82.10% with a decrease of

40.90%. This is confusing because $82.10 - 40.90 = 41.20$.

- There are no references in Table 9 on the particular precision and recall scores used for the SecureBERT 2.0 and CySecBERT rows.

8.7 Limitations of This Simulation

The results obtained in this research cannot be compared directly to the referenced benchmarks because of the following reasons, such as budget limitations for training, limited subset of data, simpler architecture (specifically, without CRF layer), differences in test data, and various computing settings. The demonstration report is brief and manually created. In order to highlight the relations, rule-based approach will be used where all actors will be connected with all mentioned techniques and tools from the report in order to extract relations. The following elements have not been implemented yet: web crawling, CRF layer, further language models, AttackER, TRAM and WAVE-27K benchmarks, validation of large language models (LLMs), model-based relation extraction, Neo4j, XAI, and adversarial robustness experiments. This research serves as a simulation in order to demonstrate how the pipeline works in practice.

9. Conclusion

The process of CTI extraction using natural language processing (NLP) is one of the key prerequisites of large-scale cybersecurity processes. The described case study describes the process of CTI extraction, including ingestion of unstructured data from OSINT, clear web, and the dark web, domain-specific adaptation of language models, named-entity recognition, relation extraction, automatic mapping to MITRE ATT&CK, and knowledge graph building based on STIX 2.1.

Research shows that domain-specific transformer models, like SecureBERT and CySecBERT solve the problem of context collapse that usually appears with general models. They score between 88 % and 94 % on named-entity recognition tasks, for example DNRTI. New multi-stage large language model pipelines that add evidence-span verification work well to connect stories to MITRE ATT&CK techniques. They deliver a Macro-F1 score than 82% while ensuring there are no false positives.

Some potential directions for research include:

1. **Real-time Graph Ingestion through Streaming:** I mix natural language processing extraction models with live network telemetry streams so that I can build threat graphs that stay current.

2. **XAI (Explainable AI):** Adding verification evidence spans into the output schema to boost confidence, in the results generated by automated systems.
3. **Adversarial NLP Robustness:** The creation of extraction models that are robust against any form of manipulation of the text, including dark web leetspeak.

References

1. Threat Intelligence Automation Using NLP and Machine Learning, <https://www.ijert.org/papers/IJCRT25A4721.pdf>
2. NLP-Based Techniques for Cyber Threat Intelligence - arXiv, <https://arxiv.org/pdf/2311.08807>
3. A Framework for Automated Cyber Threat Intelligence Extraction, <https://cmsweb.com.sg/rps2prod/ccdc2026/epro/pdf/MonA14-4.pdf>
4. SecureBERT: A Domain-Specific Language Model for Cybersecurity, https://www.researchgate.net/publication/364473594_SecureBERT_A_Domain-Specific_Language_Model_for_Cybersecurity
5. CySecBERT: A Domain-Adapted Language Model for the ... - arXiv, <https://arxiv.org/pdf/2212.02974>
6. MITRE ATT&CK Applications in Cybersecurity and The Way Forward, <https://arxiv.org/html/2502.10825v1>
7. arXiv:2408.05149v1 [cs.CR] 9 Aug 2024, <https://arxiv.org/pdf/2408.05149>
8. Multi-Step LLM Pipeline for Enhancing TTP Extraction in Cyber, https://www.researchgate.net/publication/396624793_Multi-Step_LLM_Pipeline_for_Enhancing_TTP_Extraction_in_Cyber_Threat_Intelligence
9. Natural Language Processing (NLP)-Based Frameworks for Cyber, <https://www.mdpi.com/2076-3417/16/2/619>
10. Context-aware Entity-Relation Extraction Pipeline for Threat, <https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.173627493.39916970/v1>
11. Context-aware Entity-Relation Extraction for Threat Intelligence, <https://arxiv.org/pdf/2605.15904>
12. Named Entity Recognition in Cyber Threat Intelligence Using, https://www.iti.gr/iti/wp-content/uploads/2023/11/2021_IEEE_ACTI_CSR_NER_in_CTI.pdf
13. SecureBERT 2.0: Advanced Language Model for Cybersecurity, <https://medium.com/@eaghaei/securebert-2-0-advanced-language-model-for-cybersecurity-intelligence-1b31ef60bae2>
14. Named Entity Recognition in Cyber Threat Intelligence Using, https://www.researchgate.net/publication/354415867_Named_Entity_Recognition_in_Cyber_Threat_Intelligence_Using_Transformer-based_Models
15. SoK: Automated TTP Extraction from CTI Reports – Are We There Yet?, <https://www.usenix.org/system/files/usenixsecurity25-buechel.pdf>
16. GitHub - SCreaMxp/DNRTI-A-Large-scale-Dataset-for-Named-Entity, <https://github.com/SCreaMxp/DNRTI-A-Large-scale-Dataset-for-Named-Entity-Recognition-in-Threat-Intelligence>
17. CyberNER: Multi-Type Named Entity Recognition and Alias, <https://cspub-ijcism.org/index.php/ijcism/article/download/4931/4068/11147>
18. arXiv:2302.10346v1 [cs.CL] 20 Feb 2023, <https://arxiv.org/pdf/2302.10346>

19. WAVE-27K: Bringing together CTI sources to enhance threat, <https://aclanthology.org/2024.nlpaics-1.14.pdf>
20. arXiv:2204.02685v3 [cs.CL] 20 Oct 2022, <https://arxiv.org/pdf/2204.02685>
21. CyBERT: Cybersecurity Claim Classification by Fine-Tuning ... - MDPI, <https://www.mdpi.com/2624-800X/1/4/31>
22. SecureBERT: A Domain-Specific Language Model for Cybersecurity, <https://arxiv.org/html/2204.02685v3>
23. From Words to Intelligence | The Journal on Cybercrime and Digital, <https://cyberjournal.cecyl.fr/index.php/cybin/article/view/42>
24. TTPrint: Evidence-Grounded TTP Extraction via Diverge-then ... - arXiv, <https://arxiv.org/html/2605.25836v1>
25. A Study on Improving the Automatic Classification Performance of, <https://www.preprints.org/manuscript/202510.1543>
26. Autonomous Cyber Defense Learning Using Reinforcement and, <https://www.ajol.info/index.php/drjeit/article/download/334288/314073/753278>
27. From Text to STIX: Reducing Hallucinations through Fine-Tuned LLMs, https://www.computer.org/csdl/proceedings-article/trustcom/2025/653200b030/2dMIE_FeneOA
28. A Threat Intelligence Event Extraction Conceptual Model for Cyber, <https://arxiv.org/abs/2506.03551>
29. GitHub - dessertlab/cti-to-mitre-with-nlp: Replication package for the, <https://github.com/dessertlab/cti-to-mitre-with-nlp>
30. A Study on Improving the Automatic Classification Performance of, <https://www.mdpi.com/2079-9292/14/22/4434>
31. Confidence-Qualified Threat Intelligence for Critical Infrastructure, <https://arxiv.org/html/2610.07310v1>
32. B. Jordan, R. Piazza, and T. Darley, *STIX™ Version 2.1*, OASIS Open, OASIS Standard, https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html?utm_source=chatgpt.com